

# An approximate implementation of the Bathtub Distance

Sushant Pokharel

This documentation describes the implementation process for the 'Bathtub distance' concept as a similarity heuristic. Although, it excludes grammatical gender and Honorifics as it would require manual curation of these features in the lexicon resource. Due to unavailability of ready-to-use resources and to generalize it across languages, I have limited it to character matching.

## Method Overview

This module implements a multilingual similarity and distance heuristic for comparing pairs of words across multiple languages.

CS = Characters at the start

CE = Characters at the end

1. Raw Unicode word forms
2. Longest common CS similarity
3. Longest common CE similarity
4. Grapheme-based length similarity

## Why Grapheme-Based?

A Unicode string can contain characters composed of multiple Unicode units, such as accented characters or **grapheme clusters** (Units that are perceived as a single character by a language user/learner).

### Example:

á → a + accent mark

This corresponds to two code points but a single grapheme cluster.

For this project, this distinction is important because what appears as a single written unit may internally consist of multiple Unicode components. Grapheme-based splitting ensures that such units are treated as a single comparison element.

## Grapheme Splitting Example

Using grapheme-based splitting, the following words are segmented approximately as:

शित्तल  $\rightarrow$  [शि, त्त, ल]

पित्तल  $\rightarrow$  [पि, त्त, ल]

Thus, each word is interpreted as consisting of three grapheme units, rather than multiple independent Unicode code points.

## Splitting Robustness

To truly be able to correctly split words in multiple language, the code implementation uses Regex. Specifically, `regex.findall(r"\X", text)`

It is a strong multilingual general-purpose baseline for unicode because `\X` is meant to follow Unicode grapheme-cluster rules rather than naive character splitting. The regex package documentation says its grapheme matcher conforms to the Unicode grapheme specification.

## Definitions

- $w1, w2 = \text{word1, word2}$
- $p = \text{longest common CS length}$
- $s = \text{longest common CE length}$
- $n = \text{grapheme length of } w_1$
- $m = \text{grapheme length of } w_2$

## Boundary Similarity

$$B = \frac{\min(p + s, \min(n, m))}{\min(n, m)}$$

### Step 1: Inner Minimum

Define:

$$L = \min(n, m)$$

This represents the length of the shorter word.

### Step 2: Numerator

$$\min(p + s, L)$$

This takes the total boundary match length (CS + CE) and caps it at  $L$ , ensuring it does not exceed the shorter word.

### Step 3: Final Formula

$$B = \frac{\min(p + s, L)}{L}$$

This normalizes the boundary match score to a value between 0 and 1.

### Example

Let:

$$\begin{aligned} n &= 7, & m &= 8 \\ p &= 4, & s &= 3 \end{aligned}$$

Then:

$$\begin{aligned} p + s &= 7 \\ L &= \min(7, 8) = 7 \\ B &= \frac{\min(7, 7)}{7} = \frac{7}{7} = 1 \end{aligned}$$

### Length Similarity

$$L = 1 - \frac{|n - m|}{\max(n, m)}$$

### Steps

$$\begin{aligned} \Delta &= |n - m| \\ M &= \max(n, m) \\ L &= 1 - \frac{\Delta}{M} \end{aligned}$$

### Bathtub Similarity

$$S = \frac{\alpha B + \beta L}{\alpha + \beta}$$

### Definitions

- $B$  = boundary similarity
- $L$  = length similarity
- $\alpha$  = weight for boundary similarity
- $\beta$  = weight for length similarity

## Steps

$$\text{Weighted sum} = \alpha B + \beta L$$

$$\text{Normalization} = \alpha + \beta$$

$$S = \frac{\alpha B + \beta L}{\alpha + \beta}$$

## Bathtub Distance

$$D = 1 - S$$

## Definitions

- $S$  = bathtub similarity

## Final Outputs

$$\text{Similarity} = S$$

$$\text{Distance} = 1 - S$$

Program output example on the next page:

```

● (base) sushantpokharel@Gr8Book-2 SajiloNepali % clear
● (base) sushantpokharel@Gr8Book-2 SajiloNepali % python Bathtub_distance.py "शित्तल" "पित्तल"

Words      : शित्तल / पित्तल
Similarity : 0.7333
Distance   : 0.2667
● (base) sushantpokharel@Gr8Book-2 SajiloNepali % python Bathtub_distance.py "pain" "main" तल

Words      : pain / main
Similarity : 0.8000
Distance   : 0.2000
● (base) sushantpokharel@Gr8Book-2 SajiloNepali % python Bathtub_distance.py "différence" "distance"

Words      : différence / distance
Similarity : 0.6600
Distance   : 0.3400
● (base) sushantpokharel@Gr8Book-2 SajiloNepali % python Bathtub_distance.py "مقرر" "فلتخم"

Words      : فلتخم / مقرر
Similarity : 0.1200
Distance   : 0.8800
● (base) sushantpokharel@Gr8Book-2 SajiloNepali % python Bathtub_distance.py "رابوراک" "راکوراس"

Words      : راکوراس / رابوراک
Similarity : 0.4286
Distance   : 0.5714
● (base) sushantpokharel@Gr8Book-2 SajiloNepali % python Bathtub_distance.py "cuối ngựa" "cuối ngựa"

Words      : cuối ngựa / cuối ngựa
Similarity : 1.0000
Distance   : 0.0000
● (base) sushantpokharel@Gr8Book-2 SajiloNepali % python Bathtub_distance.py "Course" "Coarse"

Words      : Course / Coarse
Similarity : 0.8667
Distance   : 0.1333
● (base) sushantpokharel@Gr8Book-2 SajiloNepali % python Bathtub_distance.py "Krankenwagen" "Krankenhaus"

Words      : Krankenwagen / Krankenhaus
Similarity : 0.6924
Distance   : 0.3076
● (base) sushantpokharel@Gr8Book-2 SajiloNepali %

```

Figure 1: Implementation example for Nepalese, French, Urdu, Arabic, Vietnamese, English and German